

Market Mayhem

Market Mayhem (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-22 · **Status:** awaiting instructor approval

Proposal (as submitted): *Market Mayhem* is a **2D top-down single-player combat racing game**. The player is in a supermarket with a **grocery list** and a **trolley**, travelling around to collect the listed items; whoever completes their list and reaches the cashier first wins. The player and **3 NPCs** can crash into one another to knock items to the floor, steal them, and immobilise each other temporarily; each item (including grocery-list items) has different effects to **boost the player or sabotage others**. Named load-bearing C++: a game loop holding a stable step with multiple shoppers each with their **own AI, inventories, physics and values** (speed, acceleration, heat gauges, etc.), running **deterministically**. Named technical features: **Enemy AI & Pathfinding** (other shoppers collect items they need or crash into the player while avoiding walls); a **real-time physics engine** handling multiple shoppers moving + crashing; **movement physics** mimicking vehicle controls (joystick, gear shifting, drifting); and **item effects** affecting runtime logic (many items, each with its own effect). Tool: a **modular effect system** — a reusable framework (e.g. a custom effects editor) to handle the real-time logic for item effects. Design features: trolleys with acceleration/braking/crashing; items with special effects; the supermarket + item aisles to collect from; and grocery lists that track collected items. Team: **6 CS + 1 Design + 1 Artist**.

Note (module completeness): the proposal doesn't name a **platform** — the joystick/vehicle controls are touch-friendly, so it's planned as a mobile build (Native Android, Pixel 9a; a Web-on-mobile build is a viable alternative). It also doesn't mention an **External Database** or accounts, but the module mandates Firebase — plan to persist progress / high scores / unlocks there (a standard, small addition; see the External Database row + M1G12-style integration). Neither is a category concern.

References:

- [Mario Kart 8 Deluxe — drifting & items guide \(Nintendo Life\)](#) (the benchmark combat kart-racer: pick up **items to boost yourself or sabotage rivals, drift for boosts**, and knock opponents aside — the item-effects + drift + combat-racing precedent)
- [Micro Machines — Wikipedia](#) (a **top-down** racer where you drive small vehicles around cluttered environments, weaving past obstacles and knocking rivals off course — the top-down combat-racing precedent)
- [Supermarket Sweep — Wikipedia](#) (the frantic **timed race to grab grocery items from supermarket aisles** — the grocery-list-race + supermarket-setting precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Market Mayhem
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — joystick + gear/drift controls are touch-first; a Web-on-mobile (Emscripten→WASM) build is a viable alternative. The Native PC build is the dev/tools vehicle (the custom effects editor). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for progress / high-score / unlock persistence.

Front end: A 2D top-down renderer — the supermarket + aisles, the player + 3 NPC trolleys, the items, crash/effect VFX, and the grocery-list + heat-gauge/effect HUD. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; some screens (menus) can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The deterministic real-time C++ engine — the fixed-timestep loop over the player + 3 NPC shoppers (each with AI, inventory, physics + values like speed/acceleration/heat), the vehicle movement + crash physics, the enemy AI + wall-avoiding pathfinding, and the modular item-effect system driving runtime logic — plus Firebase for progress/scores. **Languages:** C++ (engine); Firebase-backed services on the Extended tier.

Backend scope: Local now, server-ready (T2) — the progress/score seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the deterministic C++ loop is the heart — it advances four shoppers (each with AI, inventory, physics + heat/speed values), resolves trolley movement + crashes, and applies item effects to the runtime logic at a stable step.
- *Data-driven:* the item catalogue + their effects, the grocery lists, the supermarket layout/aisles and shopper stats (speed/accel/heat) are all data — a new item, effect or list is a data change (authored in the effects editor).
- *2D:* the top-down supermarket renderer, trolleys, items, crash/effect VFX and the grocery/heat HUD (custom-rendered). *3D:* not needed.
- *Custom tool:* the **modular effect system** / **custom effects editor** — author and wire item effects (boost/sabotage/steal/immobilise) as reusable, data-driven modules without hard-coding each in the engine.
- *Networking:* single-player (player vs 3 AI), so a Networking System is optional (●). *External DB:* progress/high-scores/unlocks (Firebase — add per the module). *Auth » / Analytics »:* module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the deterministic loop over 4 shoppers + the joystick/gear/drift input.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a deterministic fixed-timestep C++ + loop advancing the player + 3 NPC shoppers (each with AI, inventory, physics + values), with joystick + gear-shift/drift input; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the physics (M1G5) and AI (M1G8) run on its tick; the effect system (M2G2) hooks its runtime logic.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** An entity/component model with items + their effects, grocery lists, the supermarket layout/aisles and shopper stats (speed/accel/heat) in hot-reloadable JSON — a new item, effect or list is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the effects editor), M2G13 (the item-effect + inventory systems), M1G8 (AI data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (shopper AI state, inventories, physics/heat, collisions), Tracy profiling of a 4-shopper crash-heavy scene to hold the step, Catch2 tests (determinism, collision, effect application, data loading), on a GitHub Actions matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the effect-system verification.
- **G4 2D Graphics** — the top-down supermarket, trolleys, items and crash/effect VFX.

Tech & dependencies

- **Tech: Web build:** A sprite/tile renderer for the supermarket + aisles, the 4 trolleys, items, crash/effect VFX, and the grocery/heat HUD, with a camera; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2 juice/VFX; every visible frame.
- **G5 Physics & Collision** — the **vehicle movement + crashing** physics.

Tech & dependencies

- **Tech:** A C++ physics system — trolley acceleration/braking/drift (vehicle-like handling), collision + impulse for crashes that knock items loose and immobilise, broadphase for the 4 shoppers + aisles; GLM math, deterministic.
- **Prerequisites:** G1 (loop & tick), G2 (shopper stats).
- **Feeds into:** M2G11 (advanced crash/drift physics); the crash/steal/immobilise mechanics.
- **G8 Enemy AI & Pathfinding** — the **NPC shoppers** (collect items, crash the player, avoid walls).

Tech & dependencies

- **Tech:** A* / flow-field pathfinding around the aisles (wall-avoiding) with a decision layer choosing to fetch a needed item or ram the player, tuned from data; deterministic per shopper (custom C++).
- **Prerequisites:** G1 (tick), G2 (shopper/item data), G5 (movement).
- **Feeds into:** M2G10 (advanced AI — smarter tactics + difficulty); the "3 NPCs" opponents.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — hold the step under crash-heavy play + the persistence seam.

Tech & dependencies

- **Tech: Web build:** Memory pools for items/effects/particles, Tracy profiling of a 4-shopper crash-heavy scene, ASan on MSVC, and on-device validation (Pixel 9a); the Firebase/Linux-server seam (progress/scores) set up (account + software + basic connectivity test) by M2.
Native build: Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** All M1 goals (hardens the M1 codebase).
 - **Feeds into:** The Native-Android deliverable + a stable crash-heavy race.
- **G2 ★ Content Editor — Core (Modular Effects Editor)** — author item effects as reusable data-driven modules.

Tech & dependencies

- **Tech:** Dear ImGui + the shared runtime: define item effects (boost/sabotage/steal/immobilise) as composable modules with parameters, wire them to items, and round-trip JSON, with live preview and undo/redo — no hard-coding each effect in the engine.
 - **Libraries/Software:** Dear ImGui, nlohmann/json, the project's renderer (preview).
 - **Prerequisites:** G2 (item/effect data model), M1G1 (runtime hooks).
 - **Feeds into:** Fast effect/item authoring; the custom-tools showcase.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (determinism, collision, effect application, data loading) and packaging via GitHub Actions.
Native build: Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G3 (CI & test foundation).
 - **Feeds into:** Release/showcase builds of every goal.
- **G10 AI — Advanced (Smarter Shopper Tactics & Difficulty)** — the 3 NPCs matured.

Tech & dependencies

- **Tech:** Behaviour trees / utility AI weighing fetch-item vs sabotage vs flee, item/effect usage, target selection, and difficulty tiers (aggression, reaction, routing), tuned from data.
 - **Prerequisites:** M1G8 (AI & pathfinding foundation).
 - **Feeds into:** A satisfying single-player challenge; the "different behaviours" per shopper.
- **G11 Physics — Advanced (Crash & Drift Maturity)** — the crashing/ drifting deepened.

Tech & dependencies

- **Tech:** Constraint/materials + continuous collision for high-speed crashes (no tunnelling), refined drift/handling curves, and item-scatter/ knockback physics on collision; deterministic.
 - **Prerequisites:** M1G5 (physics & collision).
 - **Feeds into:** The satisfying crash/steal feel; the vehicle game-feel.
- **G13 Gameplay Systems II** — the **item-effect system + inventories + grocery list + win condition**.

Tech & dependencies

- **Tech:** C++ systems over the entity/data model: the modular item effects applied to runtime logic (boost/sabotage/steal/immobilise), per-shopper inventories, the grocery-list tracking + cashier win condition, and the heat-gauge economy; balancing-data-driven via the effects editor.
- **Prerequisites:** M1G2 (item/effect data), M1G1 (loop/runtime hooks).
- **Feeds into:** The core game depth at showcase; T2 item/mode expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 1 Art (8) — **as proposed**, and well-matched: a top-down combat racer has a contained but real art surface (the supermarket + aisles, the 4 trolleys/shoppers, the item icons, crash/effect VFX, UI) that one artist can carry, while a single designer owns the FDD, the vehicle game-feel, the item-effect catalogue + boost/sabotage balance and the AI difficulty (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the item/effect content outgrows one designer, consider a 2nd designer. *Scope note (advisory):* a deterministic 4-shopper loop + vehicle/crash physics + wall-avoiding AI + a modular effect system is a solid, well-contained single-player scope — the trolley handling + crashing, 2-3 items with effects, one supermarket and basic NPC AI is the M1/M2 floor; the full item roster, the smart per-shopper AI tactics and the polished effects editor are the stretch. The **modular effect system is the smartest lever** — it lets the team add items as data rather than engineering each; get its architecture right early.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the combat-grocery-racing USP, the trolley controls, the item-effect boost/sabotage system, the grocery-list-to-cashier win; primary benchmark e.g. Mario Kart / Micro Machines, plus a Singapore benchmark)
- M1D4 Game Feel & Touch Interaction Specification (the joystick + gear-shift + drift trolley controls, crash feel, hit-stop and feedback — as tunable data)
- M1D6 Systems & Economy Design (the item-effect catalogue — boost/sabotage/steal/immobilise — the heat-gauge economy and the balance across items — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (item/effect + AI-difficulty + handling balance via the effects editor's JSON, no code changes)

- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the grocery-list tracker, the heat-gauge/effect HUD, the minimap/aisles and the controls, plus a wireframe baseline.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — the race HUD, grocery list, results and menus navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished trolley-control + crash/effect UX tested with players on the Pixel 9a.
- **Art M1:** 1 artist (supermarket + trolleys/shoppers + items + VFX + UI)
 - M1A1 ★ Art Bible / Proof of Concept (the chaotic-supermarket visual direction — tone, palette, trolley/shopper identity language)
 - M1A2 Character Concepts & Model Sheets (the player + 3 NPC shoppers/trolleys)
 - M1A3 Environment, Props & VFX Concepts (the supermarket + aisles, the item set, and the crash/effect VFX)
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
 - M2A2 Character Animation — Extensive (trolley drift/crash, shopper reactions, immobilise/steal moments)
 - M2A5 VFX — Critical Effects & Transitions (crashes, item-effect activations, boosts, item-scatter)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (supermarket BGM, trolley engine/drift, crash, item-pickup, effect-activate and list-complete cues), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for a busy 4-shopper scene.
- **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — standard Games floor (supermarket BGM + trolley / crash / pickup / effect / sabotage / win SFX, ≥6 types), scaling with the mayhem.

- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct crash + effect cues).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the modular effects editor), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-22.*